Transport and Network Layers

Exercises 3

Computer Networks and Distributed Programming

What this pack is, and how to use it

- Thirty exercises on the transport and network layers: delays, reliable transfer, TCP, congestion, routers, IP addressing, ICMP, NAT, IPv6 and routing.
- They are for home, numbered E3.1 to E3.30: bring the number to class if one of them resists.
- Most are on paper. Three ask you to capture on your own machine what you saw in class, and one to look at your own addresses.
- There are no solutions. Where an exercise asks for a number, it also asks you to say what the number means.
- Draw time diagrams with time going down the page, one vertical line per host, as in the lectures.

Propagation and transmission

E3.1: Hosts A and B are joined by one link of rate R bps, m metres long, with propagation speed s m/s. A sends B a packet of L bits.

(a) Write the propagation delay d_{prop}, the transmission time d_{trans}, and — ignoring processing and queuing — the end-to-end delay.

(b) A starts transmitting at $t = 0$. Where is the last bit at $t = d_{\text{trans}}$?

(c) Where is the first bit at $t = d_{\text{trans}}$ if $d_{\text{prop}} > d_{\text{trans}}$? And if $d_{\text{prop}} < d_{\text{trans}}$?

(d) With $s = 2.5 \times 10^8$ m/s, $L = 1500$ bytes and $R = 10\,\text{Mbps}$, find the distance m at which $d_{\text{prop}} = d_{\text{trans}}$.

Waiting in the queue

E3.2: A packet reaches a router and is queued for its outgoing link. When it arrives, another packet is halfway through its transmission on that link, and four more are waiting. Packets leave in order of arrival.

(a) All packets are 1500 bytes and the link runs at 2.5 Mbps. What is the queuing delay of the new packet?

(b) In general: packets of length L, link rate R, x bits of the current packet already sent, n packets waiting. What is the queuing delay?

Bits in flight

E3.3: Hosts A and B are 20 000 km apart, joined by one link of $R = 5$ Mbps; the propagation speed is 2.5×10^8 m/s.

(a) Compute the bandwidth-delay product $R \cdot d_{\text{prop}}$, and say what it measures.

(b) A sends a file of 800 000 bits as one continuous message. What is the largest number of bits in the link at any moment?

(c) How long, in metres, is one bit on the link? Write the general expression in terms of s, R and the link length m.

(d) How long does the whole file take, sent continuously?

(e) Now it is sent as 20 packets of 40 000 bits, each acknowledged before the next is sent; acknowledgements take no time to transmit. How long does it take, and why is it so different from (d)?

Checksums and the alternating bit

E3.4 Take three bytes: `01010011`, `01100110`, `01110100`. Compute the one's complement of their sum, using 8-bit arithmetic with wrap-around (UDP does the same on 16 bits). Why does UDP send the complement and not the sum? How does the receiver check? Can a one-bit error go undetected? A two-bit error?

E3.5 The alternating-bit protocol — one-bit sequence numbers, ACKs, a timer — assumes the channel never reorders. Draw a time diagram, sender on the left, in which the channel reorders two messages and the receiver ends up accepting a duplicate as new data. Mark the sequence number of every data message and every ACK.

Windows

E3.6 Two hosts are joined by a 1 Gbps path with an RTT of 30 ms. Packets are 1500 bytes, headers included, and ACKs take no time to send. How many packets must the sender be allowed to have in flight for the utilization to exceed 98%?

E3.7 Go-Back-N with a sender window of 4 and sequence numbers from 0 to 1023. At time t the receiver expects packet k; the channel does not reorder. Justify both answers.

(a) Which sets of sequence numbers can be inside the sender's window at time t?

(b) Which ACK values can be travelling back to the sender at time t?

Sequence and acknowledgement numbers

E3.8: B has received from A every byte up to byte 126. A now sends two segments back to back, of 80 and 40 bytes of data. The first has sequence number 127, source port 302, destination port 80. B acknowledges every segment it receives.

(a) In the second segment: sequence number, source and destination ports?

(b) If the first segment arrives first, what are the acknowledgement number and the ports of its ACK?

(c) If the second arrives first, what is the acknowledgement number of the first ACK?

(d) Both arrive in order, but the first ACK is lost and the second arrives after the first timeout. Draw the time diagram with every segment and ACK: sequence number and data length for each segment, acknowledgement number for each ACK.

Timers and retransmissions

E3.9 Five samples arrive: `SampleRTT` = 106, 120, 140, 90 and 115 ms. Before the first, `EstimatedRTT` is 100 ms and `DevRTT` is 5 ms. With $\alpha = 0.125$ and $\beta = 0.25$, compute `EstimatedRTT`, `DevRTT` and `TimeoutInterval` after each sample.

E3.10 A sends B five segments and the second is lost; in the end all five are delivered. Timeouts are long enough for five segments and their ACKs to arrive. Consider Go-Back-N, Selective Repeat, and TCP without delayed ACKs.

(a) For each, how many segments does A send and how many ACKs does B send, and with which numbers?

(b) If the timeouts are much longer than 5 RTT, which of the three delivers all five first?

A connection you open yourself

E3.11: Two terminals on one machine, a capture on the loopback interface with display filter `tcp.port == 5001`.

```
$ nc -l 5001                        server
$ nc localhost 5001                 client:  type two short lines, then Ctrl-C
$ nc -l 12338 > copy.bin            then a file of about 200 kB
$ nc 127.0.0.1 12338 < file.bin
```

(a) Switch off relative sequence numbers. What are the two initial sequence numbers, and which options travel in the two SYNs?

(b) For each typed line: how many bytes, and what comes back?

(c) For the file: how many data segments, the largest payload, and how many data segments per ACK?

(d) Who sends the first FIN? Start again, stop the server first, then type a line in the client: what does the client get?

A congestion window, round by round

E3.12: A TCP Reno sender records its congestion window, in segments, at the start of each transmission round. Halved values are rounded down.

Round	1	2	3	4	5	6	7	8	9	10	11
cwnd	1	2	4	8	16	32	33	34	35	36	37
Round	12	13	14	15	16	17	18	19	20	21	
cwnd	38	19	20	21	22	23	1	2	4	8	

(a) In which rounds is slow start operating? Congestion avoidance?

(b) After round 12, was the loss detected by three duplicate ACKs or by a timeout? And after round 17? Justify from the table.

(c) What is the initial value of `ssthresh`? Its value in round 15? In round 20?

A congestion window, round by round (continued)

(d) In which round is the 70th segment sent?

(e) In slow start the window doubles every round, but never beyond `ssthresh`. In which state is the sender in round 22, and what is its window then?

(f) Suppose three duplicate ACKs arrive right after round 21. Using the rule of the state machine seen in the course — `ssthresh` $=$ `cwnd`$/2$, `cwnd` $=$ `ssthresh` $+3$ — what are the two values?

(g) Suppose the sender is TCP Tahoe, and the three duplicate ACKs after round 12 arrive as before. What are `ssthresh` and `cwnd` in round 15?

(h) Draw the window as a function of the round, and mark on the drawing where each loss was detected.

What stops the sender

E3.13: A sends an enormous file to B over TCP, with no loss and no timer ever expiring. A's link to the Internet runs at R bps, and the process writes into its socket at $S = 10R$ bps. B's receive buffer holds the whole file; A's send buffer holds one per cent of it.

(a) What stops the process from writing at S all the time: flow control, congestion control, or something else?

(b) Where, exactly, does the process wait, and what wakes it up?

One connection, one link without buffers

E3.14: One TCP Reno connection uses a 10 Mbps link that buffers nothing; it is the only congested link on the path. Segments are 1500 bytes, the two-way propagation delay is 150 ms, and the sender is always in congestion avoidance.

(a) What is the largest window it can reach, in segments?

(b) What are its average window and its average throughput?

(c) How long does it take to climb back to the largest window after a loss?

(d) The link can now buffer some segments. Argue that a buffer of at least $C \times RTT$, link rate times round trip, keeps the link always busy.

Two connections, one link

E3.15: Take a simplified AIMD: the window, in segments, grows by one per RTT and halves on loss (rounded down). Connections C1 and C2 share a congested link of 30 segments per second; when the offered rate exceeds it, both lose a segment. Both are in congestion avoidance. C1 has an RTT of 50 ms, C2 of 100 ms.

(a) At time t_0 both windows are 10 segments. What are they 1000 ms later?

(b) In the long run, do they get the same share of the link? Why?

(c) The course said that TCP shares a link fairly. Which assumption behind that statement fails here?

Twelve packets, four schedulers

E3.16: Twelve packets reach an output port. One packet takes one time unit to send, and a packet can start at the instant it arrives if the link is free; packet 1 is sent in $[0, 1)$.

Arrival time	0	1	2	3	5	6	8	9
Packets	1	2, 3	4	5, 6	7	8, 9	10	11, 12

For each discipline, give the time at which packets 2 to 12 leave, the delay of each from arrival to start of transmission, and the average.

(a) FIFO.

(b) Priority: odd-numbered packets are high priority.

(c) Round robin: packets 1, 2, 3, 6, 11, 12 in class 1, the rest in class 2.

(d) WFQ: odd packets in class 1 with weight 2, even in class 2 with weight 1. Explain each choice you make.

(e) What do you notice about the four averages?

A forwarding table, built and rewritten

E3.17: A router with four links forwards on 32-bit destination addresses as follows.

Link	Destination address range
0	11100000 00000000 00000000 00000000 through 11100000 00111111 11111111 11111111
1	11100000 01000000 00000000 00000000 through 11100000 01000000 11111111 11111111
2	11100000 01000001 00000000 00000000 through 11100001 01111111 11111111 11111111
3	otherwise

(a) Write a forwarding table of five entries that uses longest prefix matching and forwards correctly. Then rewrite it in `a.b.c.d/x` notation.

(b) On which link does it send each of these?

11001000 10010001 01010001 01010101
11100001 01000000 11000011 00111100
11100001 10000000 00010001 01110111

Subnetting

E3.18 A router joins three subnets whose interfaces must all have prefix `223.1.17.0/24`. Subnet 1 needs at least 60 interfaces, subnet 2 at least 90, subnet 3 at least 12. Give three network addresses, in the form `a.b.c.d/x`, that satisfy the constraints.

E3.19 Routers R1, R2 and R3 are joined in a triangle by three point-to-point links, D (R1–R2), E (R2–R3) and F (R3–R1). Each router also has a LAN: A on R1 with 250 interfaces, B on R2 with 120, C on R3 with 120.

(a) Assign the six subnets from `214.97.254.0/23`, each with enough addresses for its interfaces, routers included.

(b) Write the forwarding table of each router, using longest prefix matching.

Fragmented twice

E3.20: A host sends an IPv4 datagram of 4000 bytes — a 20-byte header without options and 3980 bytes of data — with identification 777. The first link has an MTU of 1500 bytes; further on, a router sends it over a link with an MTU of 576.

(a) List the fragments on the first link: total length, identification, offset field (in units of 8 bytes) and MF flag.

(b) List the fragments that leave the second router, with the same four fields. Who reassembles them, and where?

(c) How many header bytes reach the destination in all, against the 20 of the unfragmented datagram?

(d) One fragment is lost. What does the destination do with the others? What happens next if the datagram carried a UDP message? And a TCP segment?

A ping across the ocean — or not

E3.21: Capture with display filter `icmp` while you run:

```
$ ping -c 6 8.8.8.8
```

(a) What TTL do your requests leave with, and what TTL do the replies arrive with? Systems start from 64, 128 or 255: guess the replies' starting value and the number of routers crossed. Check the guess with `traceroute 8.8.8.8`.

(b) Take the RTT. Light in fibre covers about 2×10^8 m/s: how far away can the replying machine be? It belongs to a Californian company. Where is it, then, and how can that be?

(c) Wireshark prints the identifier and the sequence number twice, "BE" and "LE". Why would a tool need both?

(d) Ping a machine on your own LAN: which TTL comes back now?

Where ICMP lives

E3.22: ICMP messages travel inside IP datagrams, exactly as TCP segments do.

(a) Is ICMP a network-layer or a transport-layer protocol? Argue from what it does, not from where it is carried.

(b) Which two types of ICMP message does a host running `traceroute` receive, and which of the two ends the trace?

Your address, and the one the world sees

E3.23: On your own machine, run:

```
$ ip address
$ ip route
$ curl -s https://ifconfig.me ; echo
```

(a) Which address does your interface have, and which does the server report? If they differ, which box on the path made them differ, and what does it keep in memory for you?

(b) Is your own address private, shared (`100.64.0.0/10`) or public? What is your default gateway?

(c) Open a "what is my IP" page in a browser. It also names your system and browser: from which part of your request?

(d) With one pipeline, print only the names of your interfaces; then only those that are not UP.

Behind a NAT

E3.24 Every host numbers the IP datagrams it sends with an identification field that starts at a random value and then grows by one per datagram. You can capture everything a NAT sends outside.

(a) Outline a way to count the hosts behind it.

(b) Would it still work if each identification were random?

E3.25 Two peers, Arnold and Bernard, are each behind a NAT, and Arnold wants to download a file from Bernard over TCP. Devise a way for Arnold to open the connection without configuring either NAT by hand. If you cannot, explain exactly what stands in the way.

Writing IPv6 addresses

E3.26: Use the shortening rules seen in the course: leading zeros of a group may go, and one run of zero groups may become `::`.

(a) Write in the shortest legal form: `2001:0db8:0000:0000:0000:ff00:0042:8329` and `fe80:0000:0000:0000:db21:5ddb:355e:6c01`.

(b) Write in full, eight groups of four digits: `2001:db8:0:1::1` and `::1`.

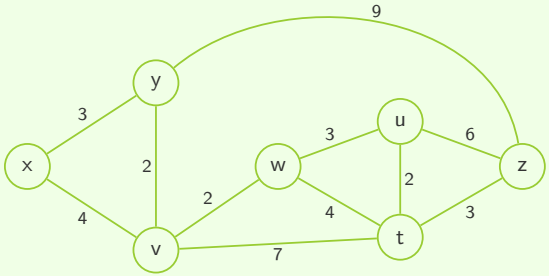
(c) Which of these is not a legal address, and why: `2001:db8::1::2`, `2001:db8:0:0:1:0:0:1`, `2001:db8:g::1`?

(d) How many `/64` subnets fit in a `/48`?

(e) Find the `inet6` lines in your `ip address`. Which one is link-local, and why would it be useless as a destination for a host on another network?

Dijkstra, by hand

E3.27: Consider this network, with the cost of each link.



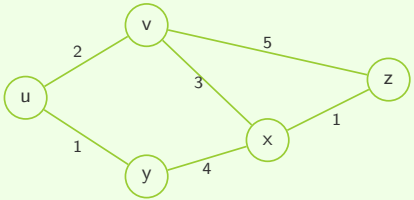
(a) Run Dijkstra's algorithm from x, writing a table like the one in the course: one row per step, with N' and $D(v), p(v)$ for every other node.

(b) Draw the tree of least-cost paths, and write the forwarding table of x.

(c) Repeat (a) and (b) from z.

Distance vectors, by hand

E3.28: Consider this network, with the cost of each link.



(a) Each node starts knowing only the costs to its neighbours. Run the distance-vector algorithm synchronously: write z's distance table after initialization and after every exchange, until nothing changes.

(b) Give z's forwarding table.

(c) The cost of link x–z rises from 1 to 10. Trace the updates that follow. Does a routing loop appear, even briefly?

Poisoned reverse

E3.29: Four routers, with link costs $c(x, y) = 4$, $c(x, z) = 50$, $c(y, w) = 1$, $c(z, w) = 1$, $c(y, z) = 3$, run distance vector with poisoned reverse.

(a) Once the tables are stable, which distances to x do w, y and z announce to each other?

(b) $c(x, y)$ rises to 60. Is there a count-to-infinity problem anyway? If so, how many iterations until it settles?

(c) How would you change $c(y, z)$ so that there is none at all when $c(x, y)$ goes from 4 to 60?

Loops and choices in BGP

E3.30: Two questions on the routing between autonomous systems.

(a) How does a BGP router detect a loop in a route it has been given?

(b) Does a BGP router always choose the loop-free route with the shortest AS-PATH? Justify with the rules of route selection seen in the course.

Tools you need

- Wireshark: as in the previous packs. For E3.11 capture on the loopback interface; for E3.21 on the one you really use.
- Terminal: `nc`, `ping`, `traceroute`, `ip`, `curl`:

```
$ sudo apt install netcat-openbsd traceroute curl iproute2
```

- Some networks drop ICMP. If `ping 8.8.8.8` gets no reply where you are, try from home: a silent network is itself an answer.
- The paper exercises need nothing but paper. For E3.12 and E3.16, a table drawn by hand is easier to read than a spreadsheet.